아키텍처 개요

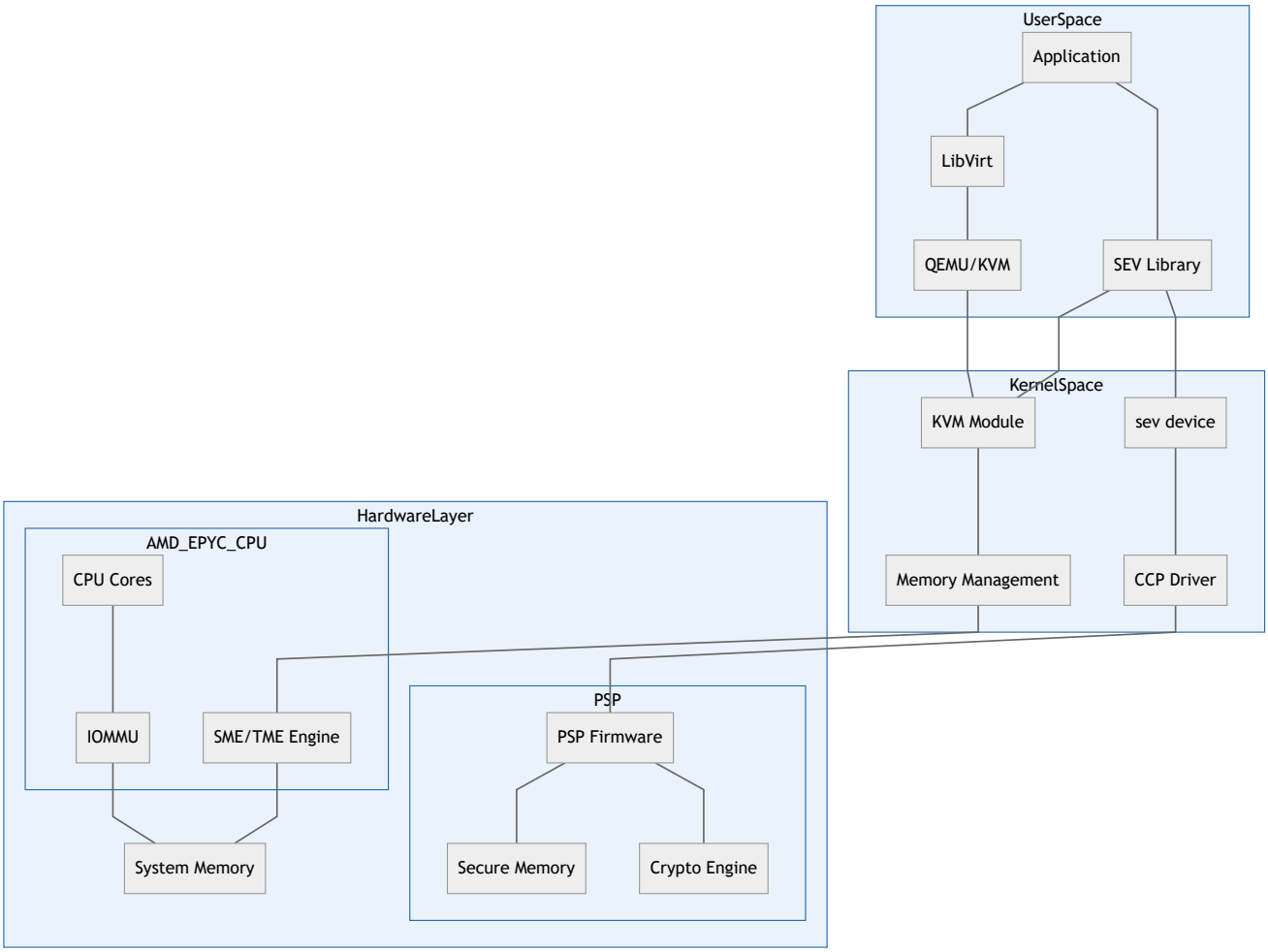
이 문서는 AMD SEV TEE의 전체 아키텍처에 대한 개요를 제공합니다.

목차

- 시스템 전체 아키텍처
- 하드웨어 계층
- 커널 계층
- 사용자 공간 계층
- 주요 컴포넌트 상호작용

시스템 전체 아키텍처

AMD SEV TEE는 하드웨어부터 사용자 애플리케이션까지 여러 계층으로 구성된 복합적인 보안 아키텍처입니다. 다음 다이어그램은 전체 시스템의 구조를 보여줍니다.



아키텍처 특징

1. **계층적 구조**: 각 계층이 명확한 역할과 책임을 가짐

2. **하드웨어 기반 보안**: PSP를 통한 하드웨어 루트 오브 트러스트

3. **투명한 암호화**: 애플리케이션 수정 없이 메모리 암호화 제공

4. **표준 인터페이스**: Linux 커널의 표준 API를 통한 접근

하드웨어 계층

하드웨어 계층은 AMD SEV의 핵심 보안 기능을 제공하는 물리적 구성요소들로 구성됩니다.

AMD EPYC 프로세서

CPU 코어

- **역할**: 게스트 VM과 호스트 시스템의 명령어 실행
- **보안 기능**:
- 메모리 태그를 통한 암호화된 메모리 식별
- VMPL (VM Permission Level) 지원 (SEV-SNP)
- 하드웨어 기반 격리 메커니즘

SME/TME 엔진 (Secure/Transparent Memory Encryption)

- **역할**: 실시간 메모리 암호화/복호화 수행
- **기능**:
- AES-128/256 암호화 알고리즘 지원
- 페이지 단위 암호화 키 관리
- 하드웨어 가속을 통한 성능 최적화
- 메모리 컨트롤러와 직접 통합

IOMMU (Input/Output Memory Management Unit)

- **역할**: I/O 디바이스의 메모리 접근 제어
- **보안 기능**:
- DMA 공격 방지
- 디바이스별 메모리 접근 권한 관리
- 게스트 VM의 I/O 격리

Platform Security Processor (PSP)

PSP는 AMD SEV의 핵심 보안 구성요소로, 독립적인 ARM Cortex-A5 프로세서입니다.

PSP 펌웨어

- **역할**: SEV 플랫폼 관리 및 보안 정책 시행
- **주요 기능**:
- 플랫폼 초기화 및 설정
- 인증서 체인 관리
- 게스트 VM 런치 승인
- 키 생성 및 관리

암호화 엔진

- **역할**: 암호화 작업 수행 및 키 관리
- **지원 알고리즘**:
- AES (Advanced Encryption Standard)
- RSA (Rivest-Shamir-Adleman)
- ECDSA (Elliptic Curve Digital Signature Algorithm)
- SHA (Secure Hash Algorithm)

보안 메모리

- **역할**: 민감한 데이터와 키 저장
- **특징**:
- 하드웨어적으로 격리된 메모리 공간
- 외부 접근 불가능
- 전원 차단 시 자동 삭제

시스템 메모리

암호화된 메모리 영역

- **구조**: 각 게스트 VM별로 독립적인 암호화 키 사용
- **관리**: Memory Encryption Engine을 통한 투명한 암호화
- **보호**: 하이퍼바이저 및 다른 게스트로부터 격리

커널 계층

Linux 커널은 하드웨어와 사용자 공간 사이의 인터페이스 역할을 수행하며, SEV 기능을 위한 여러 구성요소를 제공합니다.

KVM (Kernel-based Virtual Machine)

역할 및 기능

- **가상화 관리**: 게스트 VM의 생성, 실행, 관리
- **SEV 통합**: SEV 기능을 가상화 계층에 통합
- **메모리 관리**: 게스트 VM의 메모리 할당 및 보호

SEV 관련 기능

```
// KVM SEV 명령어 예시
#define KVM_SEV_INIT _IO(KVMIO, 0x01)
#define KVM_SEV_LAUNCH_START _IOWR(KVMIO, 0x02, struct kvm_sev_launch_start)
#define KVM_SEV_LAUNCH_UPDATE_DATA _IOWR(KVMIO, 0x03, struct kvm_sev_launch_up
#define KVM_SEV_LAUNCH_MEASURE _IOWR(KVMIO, 0x04, struct kvm_sev_launch_measur
#define KVM_SEV_LAUNCH_FINISH _IO(KVMIO, 0x05)
```

/dev/sev 디바이스

인터페이스 제공

- **플랫폼 관리**: SEV 플랫폼 설정 및 상태 관리
- **인증서 관리**: 인증서 내보내기/가져오기
- **펌웨어 통신**: PSP와의 직접 통신 인터페이스

주요 IOCTL 명령어

```
// SEV 플랫폼 관리 명령어
#define SEV_PLATFORM_RESET _IO('S', 0x01)
#define SEV_PLATFORM_STATUS _IOWR('S', 0x02, struct sev_platform_status)
#define SEV_PEK_GEN _IO('S', 0x03)
#define SEV_PEK_CSR _IOWR('S', 0x04, struct sev_pek_csr)
#define SEV_PDH_GEN _IO('S', 0x05)
#define SEV_PDH_CERT_EXPORT _IOWR('S', 0x06, struct sev_pdh_cert_export)
```

CCP (Cryptographic Coprocessor) 드라이버

역할

- **PSP 통신**: 커널과 PSP 간의 통신 중계
- **명령어 전달**: 사용자 공간 요청을 PSP로 전달
- **응답 처리**: PSP 응답을 사용자 공간으로 반환

메모리 관리 서브시스템

암호화 메모리 관리

- **페이지 태깅**: 각 메모리 페이지에 암호화 태그 할당
- **키 연결**: 메모리 페이지와 암호화 키 연결
- **접근 제어**: 권한 없는 메모리 접근 차단

사용자 공간 계층

사용자 공간에서는 다양한 구성요소들이 SEV 기능을 활용하여 보안 가상화를 구현합니다.

SEV 라이브러리 (sev crate)

모듈 구조

```
// 주요 모듈 구성
pub mod firmware; // 펌웨어 관리
pub mod launch; // 게스트 런치
pub mod certs; // 인증서 관리
pub mod measurement; // 측정 및 검증
```

```rust
pub mod session; // 세션 관리
pub mod vmsa; // VMSA 관리
```

제공 기능

1. **타입 안전성**: Rust의 타입 시스템을 활용한 안전한 API
2. **고수준 인터페이스**: 복잡한 저수준 작업을 추상화
3. **오류 처리**: 포괄적인 오류 처리 및 복구 메커니즘
4. **크로스 플랫폼**: Linux 환경에서의 일관된 동작

QEMU/KVM

역할

- **가상화 플랫폼**: 게스트 VM의 실행 환경 제공
- **SEV 통합**: SEV 기능을 가상화 스택에 통합
- **디바이스 에뮬레이션**: 가상 하드웨어 디바이스 제공

SEV 관련 기능

- 게스트 VM 메모리 암호화 설정
- 측정값 계산 및 검증
- 보안 부팅 프로세스 관리

LibVirt

관리 기능

- **VM 라이프사이클**: SEV 게스트 VM의 생성, 시작, 중지, 삭제
- **설정 관리**: SEV 관련 설정 및 정책 관리

- **모니터링**: SEV 게스트 상태 모니터링

애플리케이션

직접 사용

- SEV 라이브러리를 직접 사용하는 애플리케이션
- 커스텀 보안 요구사항을 가진 애플리케이션

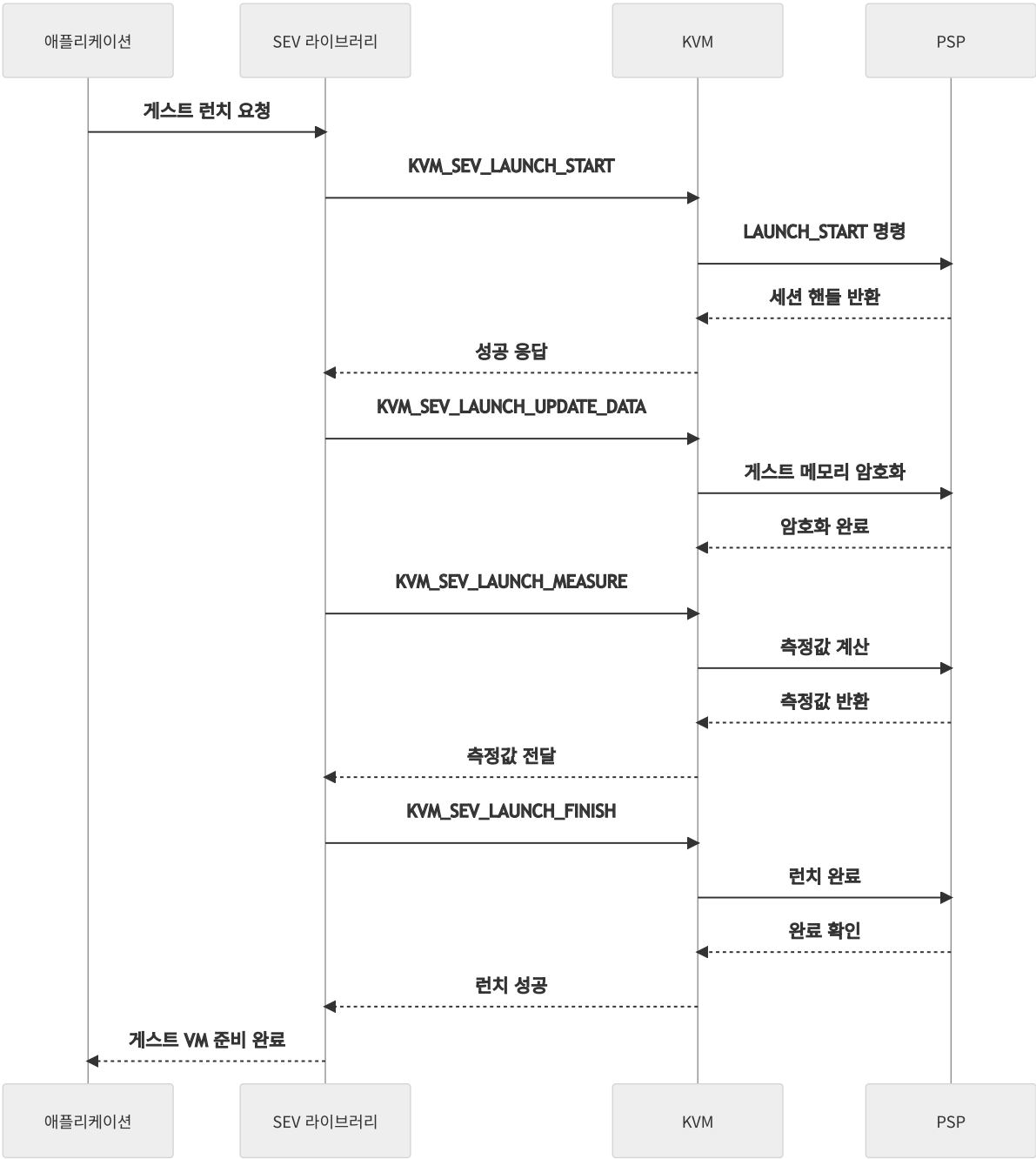
간접 사용

- LibVirt나 다른 관리 도구를 통한 SEV 기능 활용
- 클라우드 오케스트레이션 플랫폼 통합

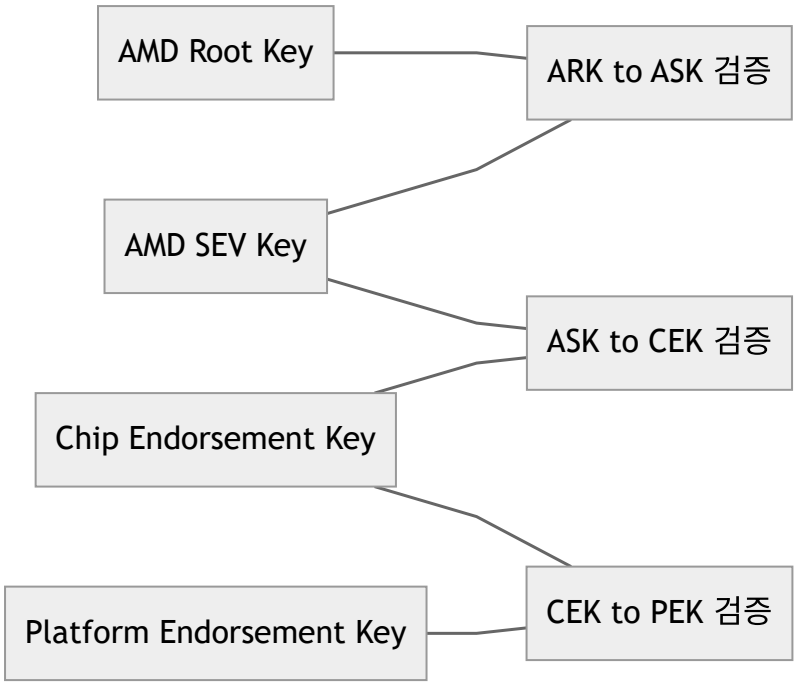
주요 컴포넌트 상호작용

AMD SEV 시스템의 각 구성요소들은 복잡한 상호작용을 통해 통합된 보안 솔루션을 제공합니다.

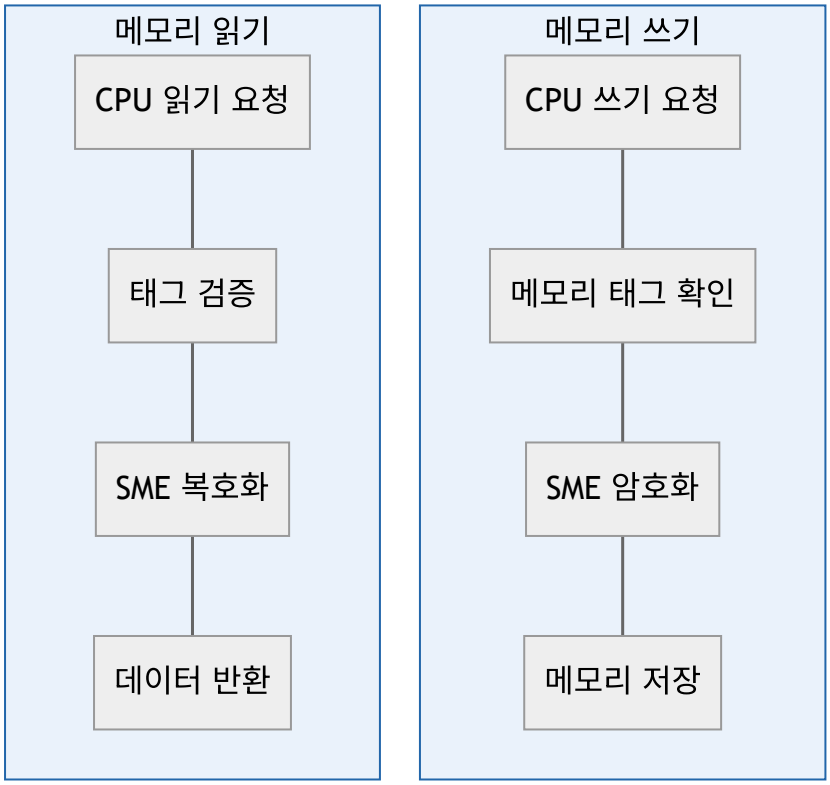
게스트 VM 런치 프로세스


애플리케이션
SEV 라이브러리
KVM
PSP
게스트 런치 요청
KVM_SEV_LAUNCH_START
LAUNCH_START 명령
세션 핸들 반환
성공 응답
KVM_SEV_LAUNCH_UPDATE_DATA
게스트 메모리 암호화
암호화 완료
KVM_SEV_LAUNCH_MEASURE
측정값 계산
측정값 반환
측정값 전달
KVM_SEV_LAUNCH_FINISH
런치 완료
완료 확인
런치 성공
게스트 VM 준비 완료
애플리케이션
SEV 라이브러리
KVM
PSP

인증서 체인 검증



메모리 암호화 플로우



데이터 플로우

1. **제어 플로우**: 사용자 공간 → 커널 → PSP

2. **데이터 플로우**: 게스트 메모리 ↔ SME ↔ 시스템 메모리

3. **보안 플로우**: PSP → 암호화 키 → SME → 메모리

성능 고려사항

최적화 요소

- **하드웨어 가속**: SME를 통한 실시간 암호화
- **캐시 효율성**: 암호화된 데이터의 캐시 활용
- **메모리 대역폭**: 암호화 오버헤드 최소화

병목 지점

- **PSP 통신**: 제어 명령어 처리 지연
- **키 관리**: 암호화 키 생성 및 교환
- **측정 계산**: 게스트 이미지 해시 계산

이러한 아키텍처를 통해 AMD SEV는 하드웨어부터 애플리케이션까지 전 계층에 걸친 포괄적인 보안 솔루션을 제공합니다.